

```

#include "server.h"
#include "server_const.h"
#include <unistd.h>

#define VERBOSE 1

extern int fildes;
static int timeout_flag = 0 ;
extern int giorno_corrente;
extern FILE *log_file;
static int primavolta = 1 ;
static int nonscattato = 1 ;
static int on_error = 1 ;
int charcnt;
int chi;
int alarm_flag ;

/*****

SERVER DEPENDENT
This is the function performing local command execution
if ( *mode ) then normal mode, else stub mode

*****/

read_data()
{
    int i ;
    char c;
    extern FILE *rs232r, *rs232w ;
    extern char client_msg[ BUF_SIZE ], device_msg[ BUF_SIZE ] ;
    i=0;
    do
    {
        c = fgetc( rs232r ) ;
        device_msg[i] = c ;
        charcnt = i ;
        i++ ;
    } while ( c != '\r' && on_error != 1 ) ;
    device_msg[i] = 0 ;
    device_msg[i-1] = 0 ;
    device_msg[i-2] = 0 ;
}

int ps_do_read()
{
    int i ;
    char c ;

    extern char client_msg[ BUF_SIZE ], device_msg[ BUF_SIZE ],
                myname[ NCHARS ] ;
    extern FILE *rs232r, *rs232w ;
    extern int alarm_flag ;

    extern int verbose, fildes;
    static struct timeval timeout = {(long)3, (long)0};
    fd_set ready;
    int return_val;

```

```

/* system call che serve per la gestione del TIMEOUT */
FD_ZERO(&ready);

//printf("in ps_do_read\n");

if (alarm_flag){
    i = 0 ;
    do {
        if (alarm_flag) {

            FD_SET(fildes, &ready);
            timeout.tv_sec = 2 ;
            timeout.tv_usec = 0 ;

printf("*1*\n");
            if ( return_val = select( getdtablesize(), &ready,
                                   (fd_set *)NULL, (fd_set *)NULL,
                                   &timeout) > 0 ) {

                device_msg[i] = (char) NULL ;
                c = fgetc( rs232r ) ;

printf("*2*\n");
                if ( c != '\r' && c != '\0' )
                    device_msg[i] = c ;
                else
                    device_msg[i] = ' ';
                FD_CLR(fildes, &ready);
            }
            else {
                alarm_flag = FALSE ;
                printf( "\n%s timeout occured ^G\n", myname ) ;
                system("date") ;

printf("*3*\n");
            }

        }
        //printf( "PSD i=%d c=%c=\n", i, c ) ;
        i++ ;
printf("*4*\n");
    } while ( (c != '\r') && (alarm_flag > 0) ) ;
    //device_msg[i] = '\0' ;
    //device_msg[i-1] = '\0' ;
    //strcpy(device_msg, device_msg+1);

}
return (alarm_flag) ;
}

exec_command( mode, flag_pointer )
int *mode, *flag_pointer ;
{
extern int fildes ;
extern int main_socket;
extern int verbose, alarm_enabled, alarm_allowed ;
extern char client_msg[ BUF_SIZE ], device_msg[ BUF_SIZE ] ;
extern char myname[ NCHARS ], alarm_headmsg[ NCHARS ] ;
extern char tty_name[ NCHARS ] ;
extern FILE *rs232r, *rs232w ;
extern struct supply_data supply[100];
extern int address[100];
extern int alim_mode[100];
extern int supply_number;

```

```

extern char *message;
int return_val;
time_t t;
struct tm *tempo;

int i ;
char c;

static int out_cur = 0, tesla = 0, out_volt = 0,
           int15pos = 0, int15neg = 0, int5pos = 0,
           delta_t = 0, vce = 0, opt_iout = 0, aux_iout = 0;

/* for stub mode */
static int a = 0, p = 2048, kf = 1024, f = 25 ;

static struct timeval timeout = {(long)1, (long)0};
static struct timeval timeout1 = {(long)0, (long)10000};
fd_set ready;

int status, loop_power, locked ;
char command, answer[ BUF_SIZE ] ;

extern int timeout_flag ;
extern void on_timeout() ;

int rem_loc, retry;

int line_ok;

char risp[100];
char file_daaprire[100];

FD_ZERO(&ready);

//    if ( verbose )
//printf( "\n%s: command is %s", myname, client_msg ) ;

message = client_msg + 10;

sscanf ( client_msg, "ALIM = %d", &supply_number);

sscanf( message, "%c", &command ) ;

switch ( command )
{
    case EOC :
        *flag_pointer = FALSE ;
        break;

    case ALARMS_ON :
        if ( alarm_allowed )
            alarm_enabled = TRUE ;
        else
            strcpy( device_msg, NEG_ANSWER ) ;
        break;

    case ALARMS_OFF :
        alarm_enabled = FALSE ;

```

```

break;

case ALLADCS:
device_msg[0] = 0 ;
if ( alim_mode[supply_number])
    read_alladcs() ;
else
{
    strcpy( device_msg, "AD 0 = 999 " ) ;
    strcat( device_msg, "AD 2 = 999 " ) ;
    strcat( device_msg, "AD 3 = 999 " ) ;
    strcat( device_msg, "AD 4 = 999 " ) ;
    strcat( device_msg, "AD 5 = 999 " ) ;
    strcat( device_msg, "AD 6 = 999 " ) ;
    strcat( device_msg, "AD 7 = 999 " ) ;
}
break ;

case RESET:
if ( *mode )
    tty_reset() ;
else
    strcpy( device_msg, " stub device reset " ) ;
break ;

case DOIT:

/* ma server specific message execution: first char must
   be '!' */
/* send the message */

if ( alim_mode[supply_number] )
{
    t = time( (long*) 0);
    tempo = localtime (&t);
    if ( giorno_corrente != tempo->tm_yday )
    {
        fflush(log_file);
        fclose(log_file);
        sprintf(file_daaprire, "./logs/server%d_%d.log", main_socket,
tempo->tm_yday);
        giorno_corrente = tempo->tm_yday;
        log_file = fopen (file_daaprire, "a");
    }
    fprintf(log_file,"%s\n", message+1 );

    fprintf( rs232w, "ADR %dr",address[supply_numbe
r]);

    fflush( rs232w ) ;

    if (
        ( ( strcmp ( message , "!WA", 3 ) == 0 ) ||
          ( strcmp ( message , "!N", 2 ) == 0 ) ||
          ( strcmp ( message , "!RS", 3 ) == 0 ) ||

```

```

        ( strcmp ( message , "IF", 2 ) == 0 ) )
    )
    {
        on_error = 0 ;
        i = 0 ;
        primavolta = 1 ;
        do
        {
            for ( i = 0 ; i < BUF_SIZE ; i++ ) device_msg[i]
= 0 ;

            on_error = 0 ;

            if ( alim_mode[supply_number] == 0 ) break;
            fprintf( rs232w, "CMDSTATE\r" ) ;
            fflush( rs232w ) ;

            for ( i = 0 ; i < BUF_SIZE ; i++ ) device_msg[i]
= 0 ;

            i = 0 ;

            usleep ( 50000 ) ;
            alarm_flag = TRUE;
            ps_do_read();

            i = -1 ;
            do
            {
                i++;
                if ( device_msg[i] == '\n' ) line_ok = 1

                else line_ok = 0 ;
            } while (device_msg[i] != '\n' && ( i !
= BUF_SIZE - 1 ) ) ;

            device_msg[i] = '\0' ;
            device_msg[i+1] = '\0' ;

            if ( line_ok == 0 ) on_timeout();

        } while ( on_error ) ;

        if ( strcmp ( device_msg , "LOCAL", 5 ) == 0 ) rem_loc
        if ( strcmp ( device_msg , "REMOTE", 6 ) == 0 ) rem_lo

    }

    if (
    ( ( strcmp ( message , "IWA", 3 ) != 0 ) &&
    ( strcmp ( message , "IN", 2 ) != 0 ) &&
    ( strcmp ( message , "IRS", 3 ) != 0 ) &&
    ( strcmp ( message , "IF", 2 ) != 0 ) ) ||

    ( ( ( strcmp ( message , "IWA", 3 ) == 0 ) ||
    ( strcmp ( message , "IN", 2 ) == 0 ) ||
    ( strcmp ( message , "IRS", 3 ) == 0 ) ||
    ( strcmp ( message , "IF", 2 ) == 0 ) ) && (rem_loc ==
REMOTE) ) )
    )

```

```

{
    // sono le domande che prevedono una risposta

// =====
=====

do
    primavolta = 1 ;
    {
        for ( i = 0 ; i < BUF_SIZE ; i++ ) device_msg[i] = 0 ;

        i = 0 ;

        on_error = 0 ;
        if ( alim_mode[supply_number] == 0) break;
        //if ( VERBOSE ) printf("2 ** primavolta = %d on
        _error = %d, mode = %d, addr = %d\n",
        //
        address[supply_number]);

        fflush( rs232w ) ;
        fflush( rs232r ) ;
        fprintf( rs232w, "ADR %dr", address[supply_numbe
r]);

        fflush( rs232w ) ;
        fprintf( rs232w, "%s\r", ( message + 1 ) ) ; /*
per saltare il ! */

        fflush( rs232w ) ;

        if ( ( strcmp ( message , "!AD", 3 ) == 0 ) ||
            ( strcmp ( message , "!RA", 3 ) == 0 ) ||
            ( strcmp ( message , "!WA", 3 ) == 0 ) ||
            ( strcmp ( message , "!CMDSTATE", 9 ) == 0 ) |
            ( strcmp ( message , "!S1", 3 ) == 0 ) )

            {
                /* get the answer */
                i = 0 ;

                for ( i = 0 ; i < BUF_SIZE ; i++ ) device
                i = 0 ;

                usleep ( 50000 ) ;
                alarm_flag = TRUE;
                ps_do_read();

                i = -1 ;
                do
                {
                    i++;
                    if ( device_msg[i] == '\n' ) alim_mode[s
                    else alim_mode[supply_number] = 0 ;
                } while (device_msg[i] != '\n' ) ;
                device_msg[i] = '\0' ;
                device_msg[i+1] = '\0' ;

supply_number] = 1 ;

```

```
}
```

```
} while ( on_error );
```

```
// =====  
=====
```

```
}
```

```
else
```

```
{  
    // domande che non prevedono risposta  
    if ( VERBOSE ) printf("-----in else !!\n");  
    sprintf ( device_msg, "MA SERVER DONE" ) ;  
}
```

```
}
```

```
else
```

```
{
```

```
/* DO IT STUB MODE */
```

```
if ( strcmp ( message , "!AD 0" ) == 0 )  
{  
    sprintf ( device_msg, "%3d", out_cur ) ;  
    out_cur++;  
    if ( out_cur > 999 ) out_cur = 0 ;  
}
```

```
if ( strcmp ( message , "!AD 1" ) == 0 )  
{  
    sprintf ( device_msg, "%3d", tesla ) ;  
    tesla++;  
    if ( tesla > 999 ) tesla = 0 ;  
}
```

```
if ( strcmp ( message , "!AD 2" ) == 0 )  
{  
    sprintf ( device_msg, "%3d", out_volt ) ;  
    out_volt++;  
    if ( out_volt > 999 ) out_volt = 0 ;  
}
```

```
if ( strcmp ( message , "!AD 3" ) == 0 )  
{  
    sprintf ( device_msg, "%3d", int15pos ) ;  
    int15pos++;  
    if ( int15pos > 999 ) int15pos = 0 ;  
}
```

```
if ( strcmp ( message , "!AD 4" ) == 0 )  
{  
    sprintf ( device_msg, "%3d", int15neg ) ;  
    int15neg++;  
    if ( int15neg > 999 ) int15neg = 0 ;  
}
```

```
if ( strcmp ( message , "!AD 5" ) == 0 )  
{  
    sprintf ( device_msg, "%3d", int5pos ) ;  
    int5pos++;  
    if ( int5pos > 999 ) int5pos = 0 ;  
}
```

```

if ( strcmp ( message , "!AD 6" ) == 0 )
{
    sprintf ( device_msg, "+%2d", delta_t ) ;
    delta_t++;
    if ( delta_t > 99 ) delta_t = 0 ;
}

if ( strcmp ( message , "!AD 7" ) == 0 )
{
    sprintf ( device_msg, "%3d", vce ) ;
    vce++;
    if ( vce > 999 ) vce = 0 ;
}

if ( strcmp ( message , "!AD 8" ) == 0 )
{
    sprintf ( device_msg, "%5d", opt_iout ) ;
    opt_iout++;
    if ( opt_iout > 99999 ) opt_iout = 0 ;
}

if ( strcmp ( message , "!AD 9" ) == 0 )
{
    sprintf ( device_msg, "%3d", aux_iout ) ;
    aux_iout++;
    if ( aux_iout > 999 ) aux_iout = 0 ;
}

if ( strcmp ( message , "!ADR" ) == 0 )
{
    sprintf ( device_msg, "%d", 255 ) ;
}

if ( strcmp ( message , "!ASW" ) == 0 )
{
    sprintf ( device_msg, "%s", "R" ) ;
}

if ( strcmp ( message , "!BAUD" ) == 0 )
{
    sprintf ( device_msg, "%d", 9600 ) ;
}

if ( strcmp ( message , "!CMD" ) == 0 )
{
    sprintf ( device_msg, "%s", "REM" ) ;
}

if ( strcmp ( message , "!CMDSTATE" ) == 0 )
{
    sprintf ( device_msg, "%s", "STUB" ) ;
}

if ( strcmp ( message , "!ERRC" ) == 0 )
{
    sprintf ( device_msg, "MA SERVER DONE" ) ;
}

if ( strcmp ( message , "!ERRT" ) == 0 )
{
    sprintf ( device_msg, "MA SERVER DONE" ) ;
}

```



```
if ( strcmp ( message , "!F" ) == 0 )
{
    sprintf ( device_msg, "MA SERVER DONE" ) ;
}

if ( strcmp ( message , "!LALL" ) == 0 )
{
    sprintf ( device_msg, "MA SERVER DONE" ) ;
}

if ( strcmp ( message , "!LOC" ) == 0 )
{
    sprintf ( device_msg, "MA SERVER DONE" ) ;
}

if ( strcmp ( message , "!LOCK" ) == 0 )
{
    sprintf ( device_msg, "MA SERVER DONE" ) ;
}

if ( strcmp ( message , "!IN" ) == 0 )
{
    sprintf ( device_msg, "MA SERVER DONE" ) ;
}

if ( strcmp ( message , "!NASW" ) == 0 )
{
    sprintf ( device_msg, "MA SERVER DONE" ) ;
}

if ( strcmp ( message , "!NERR" ) == 0 )
{
    sprintf ( device_msg, "MA SERVER DONE" ) ;
}

if ( strcmp ( message , "!PO" ) == 0 )
{
    sprintf ( device_msg, "+" ) ;
}

if ( strcmp ( message , "!PO +" ) == 0 )
{
    sprintf ( device_msg, "MA SERVER DONE" ) ;
}

if ( strcmp ( message , "!PO -" ) == 0 )
{
    sprintf ( device_msg, "MA SERVER DONE" ) ;
}

if ( strcmp ( message , "!RA" ) == 0 )
{
    sprintf ( device_msg, "%6d", 999999 ) ;
}

if ( strcmp ( message , "!REM" ) == 0 )
{
    alim_mode[supply_number] = 1 ;
    sprintf ( device_msg, "MA SERVER DONE" ) ;
}

if ( strcmp ( message , "!RS" ) == 0 )
{

```

```

        sprintf ( device_msg, "MA SERVER DONE" ) ;
    }

    if ( strcmp ( message , "!S1" ) == 0 )
    {
        sprintf ( device_msg, "STUB" ) ;
    }

    if ( strcmp ( message , "!UNLOCK" ) == 0 )
    {
        sprintf ( device_msg, "MA SERVER DONE" ) ;
    }

    if ( strncmp ( message , "!WA", 3 ) == 0 )
    {
        sprintf ( device_msg, "STUB" ) ;
    }

    if ( verbose ) printf( "%s -> %24s\n", tty_name, device_msg ) ;
}
break ;
default:
printf( "\n%s: unknown command\n", myname ) ;
strcpy( device_msg, NEG_ANSWER ) ;
break ;
}
}

/*****

SERVER DEPENDENT
This is the function performing timeout actions when /dev/tty does
not answer.

*****/

void on_timeout()
{

extern char tty_name[ NCHARS ] ;
extern int main_socket;
extern int address[100];
char filename[20];
char comando[20];

extern int fildes ;
extern FILE *rs232r, *rs232w ;
    extern int verbose ;
    extern char client_msg[ BUF_SIZE ], device_msg[ BUF_SIZE ] ;
    extern char myname[ NCHARS ] ;
    extern char *message;
    extern int timeout_flag, mode ;
    extern int supply_number;
    extern int alim_mode[100];
    FILE *timeout_file;

fflush( stdout ) ;
timeout_file = fopen("timeout_file", "a");
//printf("IN TIMEOUT: primavolta = %d on_error = %d, mode = %d, addr = %d\n", pr
imavolta, on_error, alim_mode[supply_number], address[supply_number]) ;
fflush( stdout ) ;

```

```

timeout_flag = 1 ;
if ( primavolta )
{
    primavolta = 0 ;
    on_error = 1 ;
    fprintf(timeout_file, "S K I P %s alim = %d ** %s ** i = %d %d ", tty_name,  address[supply_number], device_msg,  charcnt, chi);
    printf("errore skippato su %s alim = -sk-%d-\n\n\n", tty_name,  address[supply_number]);
    fclose(timeout_file);
    system("date >> timeout_file");

    fprintf( rs232w, "\r" ) ;
    fflush( rs232w ) ;
    sleep(1);
    fprintf( rs232w, "\r" ) ;
    fflush( rs232w ) ;
    sleep(1);
    close(fildes);
    sleep(1);
    tty_init();
}
else
{
    alim_mode[supply_number] = 0 ;
    on_error = 1 ;
    fprintf(timeout_file, "S T U B %s alim = %d ** %s ** i = %d %d ", tty_name,  address[supply_number], device_msg,  charcnt, chi);
    printf("S T U B : nlim= %d, addr= -S-%d- line is %s ** %s **\n\n\n", supply_number, address[supply_number], tty_name, device_msg);
    fclose(timeout_file);
    system("date >> timeout_file");
    sprintf ( device_msg, "STUB" ) ;
}
}

```